

Projekt nr 3 - PUS

Etap 2: Projekt aplikacji wykorzystującej protokół RCMP - RCMP Chat – Secure Real-Time Messaging System

Bartłomiej Źądło, nr albumu: 151505

1. Opis funkcjonalny aplikacji

1.1. Cel aplikacji

RCMP Chat to aplikacja komunikatora czasu rzeczywistego wykorzystująca autorski protokół RCMP (Real-Time Chat Messaging Protocol). System umożliwia bezpieczną wymianę wiadomości tekstowych pomiędzy użytkownikami w ramach pokoi tematycznych oraz wiadomości prywatnych 1:1.

Aplikacja została zaprojektowana jako demonstracja praktycznego wykorzystania autorskiego protokołu sieciowego uwzględniającego:

- bezpieczną komunikację,
- niezawodne dostarczanie wiadomości,
- zarządzanie sesjami użytkowników,
- odporność na utratę połączenia,
- wykrywanie duplikatów wiadomości,
- obsługę błędów i retry.

1.2. Problem rozwiązywany przez aplikację

Aplikacja rozwiązuje problem bezpiecznej i niezawodnej komunikacji tekstowej w czasie rzeczywistym pomiędzy wieloma użytkownikami.

Typowe proste rozwiązania socketowe nie zapewniają:

- autoryzacji użytkowników,
- odporności na utratę połączenia,
- wykrywania duplikatów wiadomości,
- retransmisji danych,
- integralności komunikatów,
- ochrony przed replay attack,
- mechanizmów keep-alive.

RCMP Chat implementuje własny protokół aplikacyjny zapewniający powyższe mechanizmy.

1.3. Użytkownicy systemu

Użytkownik komunikatora

Może:

- logować się do systemu,
- dołączać do pokoiów,
- wysyłać wiadomości,
- odbierać wiadomości,
- zmieniać status obecności,
- komunikować się prywatnie z innymi użytkownikami.

Administrator serwera

Odpowiada za:

- konfigurację serwera,
- zarządzanie pokojami prywatnymi,
- monitorowanie logów i błędów,
- kontrolę bezpieczeństwa systemu.

2. Architektura rozwiązania

2.1. Model architektury

Aplikacja wykorzystuje model klient–serwer.

Wszystkie wiadomości przechodzą przez centralny serwer RCMP. Klienci nie komunikują się bezpośrednio między sobą.

Model klient–serwer umożliwia:

- centralne zarządzanie autoryzacją,
- kontrolę dostępu do pokoiów,
- wykrywanie duplikatów wiadomości,
- realizację rate limiting,
- logowanie aktywności użytkowników,
- centralne zarządzanie bezpieczeństwem.

2.2. Komponenty systemu

Klient RCMP

Aplikacja użytkownika odpowiedzialna za:

- logowanie,
- utrzymywanie sesji,
- wysyłanie wiadomości,
- odbieranie wiadomości,
- reconnect po utracie połączenia,
- obsługę PING/PONG,
- retransmisję wiadomości.

Klient może zostać zaimplementowany jako aplikacja CLI lub desktopowa.

Serwer RCMP

Centralny komponent systemu odpowiedzialny za:

- obsługę połączeń TCP/TLS,
- walidację komunikatów RCMP,
- autoryzację użytkowników,
- routing wiadomości,
- zarządzanie pokojami,
- retransmisję i ACK,
- wykrywanie timeoutów,
- ochronę przed nadużyciami.

Moduł autoryzacji

Odpowiada za:

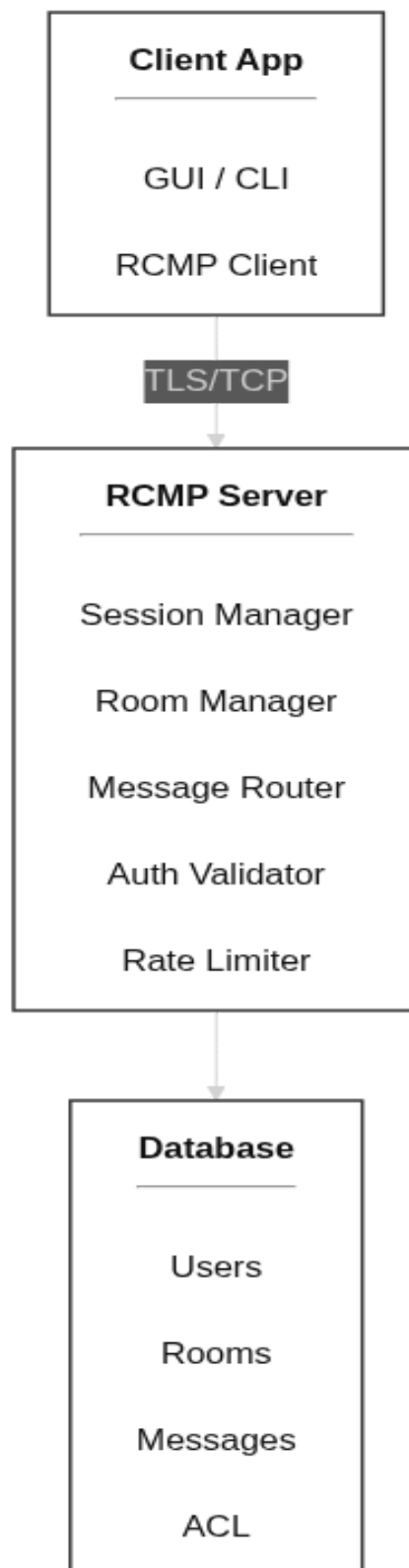
- weryfikację loginu i hasła,
- generowanie tokenów JWT,
- zarządzanie sesjami,
- walidację tokenów,
- kontrolę dostępu do pokoi prywatnych.

Baza danych (Postgre SQL)

Przechowuje:

- użytkowników,
- hash haseł bcrypt,
- pokoje,
- listy ACL pokoi prywatnych,
- historię wiadomości,
- logi systemowe.

2.3. Diagram komponentów



2.4. Przepływ danych

1. Klient nawiązuje połączenie TCP/TLS z serwerem.
2. Klient wysyła komunikat LOGIN.
3. Serwer weryfikuje dane użytkownika.
4. Serwer odsyła LOGIN_OK wraz z tokenem JWT.
5. Klient może dołączać do pokoiów oraz wysyłać wiadomości.
6. Serwer przekazuje wiadomości odbiorcom poprzez DELIVER_MESSAGE.
7. Odbiorcy potwierdzają odbiór przez MESSAGE_ACK.
8. Po zakończeniu sesji klient wysyła BYE.

3. Przypadki użycia

UC1 – Logowanie użytkownika

Cel

Uzyskanie autoryzowanej sesji użytkownika.

Aktor

Użytkownik.

Warunki wstępne

- aktywne połączenie TCP/TLS,
- użytkownik posiada konto.

Scenariusz główny

1. Użytkownik uruchamia klienta.
2. Wprowadza login i hasło.
3. Klient wysyła komunikat LOGIN.
4. Serwer weryfikuje dane.
5. Serwer generuje token JWT.
6. Serwer wysyła LOGIN_OK.
7. Sesja przechodzi do stanu AUTHENTICATED.

Scenariusze alternatywne

- błędne hasło → LOGIN_ERR 4011,
- przekroczony limit logowań → LOGIN_ERR 4012,
- timeout połączenia → rozłączenie klienta.

Wynik końcowy

Użytkownik posiada aktywną sesję.

UC2 – Dołączenie do pokoju

Cel

Dołączenie użytkownika do wybranego pokoju.

Aktor

Użytkownik.

Warunki wstępne

- użytkownik jest zalogowany,
- pokój istnieje.

Scenariusz główny

1. Klient wysyła JOIN_ROOM.
2. Serwer sprawdza uprawnienia.
3. Serwer dodaje użytkownika do pokoju.
4. Serwer rozsyła ROOM_EVENT do pozostałych użytkowników.

Scenariusze alternatywne

- pokój nie istnieje → ERROR 4041,
- brak dostępu → ERROR 4032.

Wynik końcowy

Użytkownik należy do pokoju.

UC3 – Wysłanie wiadomości do pokoju

Cel

Dostarczenie wiadomości do wszystkich użytkowników pokoju.

Aktor

Użytkownik.

Warunki wstępne

- użytkownik należy do pokoju,
- sesja jest aktywna.

Scenariusz główny

1. Użytkownik wpisuje wiadomość.
2. Klient generuje SEND_MESSAGE.
3. Klient oblicza HMAC wiadomości.
4. Serwer weryfikuje HMAC.
5. Serwer zapisuje wiadomość.
6. Serwer rozsyła DELIVER_MESSAGE.
7. Odbiorcy odsyłają MESSAGE_ACK.

Scenariusze alternatywne

- invalid HMAC → ERROR 4005,
- przekroczony rate limit → ERROR 4291,
- utrata połączenia → retransmisja wiadomości,
- duplikat msg_id → ACK bez ponownego dostarczenia.

Wynik końcowy

Wiadomość została dostarczona do użytkowników pokoju.

UC4 – Wiadomość prywatna 1:1

Cel

Wysłanie prywatnej wiadomości do konkretnego użytkownika.

Aktor

Użytkownik.

Warunki wstępne

- obaj użytkownicy są zalogowani.

Scenariusz główny

1. Klient wysyła SEND_MESSAGE z identyfikatorem odbiorcy.
2. Serwer odnajduje odbiorcę.
3. Serwer przesyła DELIVER_MESSAGE do odbiorcy.
4. Odbiorca odsyła MESSAGE_ACK.

Scenariusze alternatywne

- użytkownik nie istnieje → ERROR 4042,
- odbiorca offline → wiadomość może zostać zapisana w historii.

Wynik końcowy

Prywatna wiadomość została dostarczona.

UC5 – Utrata połączenia i reconnect

Cel

Przywrócenie sesji po utracie połączenia.

Aktor

Użytkownik.

Warunki wstępne

- aktywna sesja użytkownika.

Scenariusz główny

1. Klient przestaje otrzymywać PONG.
2. Klient wykrywa timeout.
3. Klient rozpoczyna reconnect.
4. Klient ponownie wykonuje LOGIN.
5. Serwer odtwarza stan pokojów użytkownika.
6. Sesja zostaje przywrócona.

Scenariusze alternatywne

- token JWT wygaś → wymagane ponowne logowanie,
- serwer niedostępny → exponential backoff.

Wynik końcowy

Połączenie zostaje odzyskane.

UC6 – Próba wejścia do prywatnego pokoju bez uprawnień

Cel

Weryfikacja mechanizmów autoryzacji.

Aktor

Użytkownik.

Warunki wstępne

- użytkownik jest zalogowany.

Scenariusz główny

1. Klient wysyła JOIN_ROOM.
2. Serwer sprawdza ACL pokoju.
3. Serwer odrzuca żądanie.
4. Serwer wysyła ERROR 4032.

Scenariusze alternatywne

- użytkownik otrzymał zaproszenie → dołączenie do pokoju.

Wynik końcowy

Dostęp do pokoju został poprawnie zablokowany.

UC7 – Zarządzanie pokojami przez administratora

Cel

Umożliwienie administratorowi tworzenia, modyfikacji oraz usuwania pokoi komunikacyjnych w systemie RCMP Chat.

Aktor

Administrator systemu.

Warunki wstępne

- administrator jest zalogowany (LOGIN + ważny JWT),
- posiada rolę ADMIN w tokenie,
- serwer RCMP jest aktywny.

Scenariusz główny

Tworzenie pokoju:

1. Administrator wysyła żądanie utworzenia pokoju (np. CREATE_ROOM).
2. Serwer weryfikuje token JWT i uprawnienia ADMIN.
3. Serwer tworzy nowy pokój w bazie danych.
4. Serwer inicjalizuje listę uczestników (pusta lub z ACL).
5. Serwer odsyła potwierdzenie operacji (ROOM_CREATED / SUCCESS).

Modyfikacja lub usunięcie pokoju:

1. Administrator wysyła UPDATE_ROOM lub DELETE_ROOM.
2. Serwer sprawdza istnienie pokoju.
3. Serwer wykonuje operację.
4. Użytkownicy aktualnie w pokoju otrzymują ROOM_EVENT lub ERROR (np. pokój usunięty).
5. Serwer potwierdza wykonanie operacji.

Scenariusze alternatywne

- brak uprawnień → ERROR 4032 FORBIDDEN,
- pokój nie istnieje → ERROR 4041 ROOM_NOT_FOUND,
- próba usunięcia systemowego pokoju → ERROR 4032 FORBIDDEN,
- błąd serwera → ERROR 5001 SERVER_ERROR,
- przeciążenie → ERROR 5002 SERVER_OVERLOAD.

Wynik końcowy

Stan systemu zostaje zaktualizowany (pokój utworzony / zmodyfikowany / usunięty), a zmiany są propagowane do aktywnych klientów.

UC8 – Zarządzanie użytkownikami przez administratora

Cel

Zapewnienie kontroli nad użytkownikami systemu poprzez możliwość blokowania, odblokowywania oraz wymuszania zakończenia sesji.

Aktor

Administrator systemu.

Warunki wstępne

- administrator jest zalogowany,
- posiada rolę ADMIN,
- użytkownik docelowy istnieje w systemie.

Scenariusz główny

Blokada użytkownika:

1. Administrator wysyła żądanie BLOCK_USER.
2. Serwer weryfikuje uprawnienia ADMIN.
3. Serwer oznacza konto użytkownika jako zablokowane.
4. Wszystkie aktywne sesje użytkownika zostają zakończone (BYE + ERROR 4032).
5. Użytkownik nie może ponownie się zalogować.

Odblokowanie użytkownika:

1. Administrator wysyła UNBLOCK_USER.
2. Serwer aktualizuje status konta.
3. Użytkownik odzyskuje możliwość logowania.

Wymuszenie rozłączenia:

1. Administrator wysyła FORCE_DISCONNECT.
2. Serwer natychmiast zamyka aktywne połączenie TCP.
3. Klient otrzymuje BYE / ERROR 4031 SESSION_TERMINATED.

Scenariusze alternatywne

- użytkownik nie istnieje → ERROR 4042 USER_NOT_FOUND,
- brak uprawnień → ERROR 4032 FORBIDDEN,
- użytkownik już zablokowany → ERROR 4091 ALREADY_BLOCKED,
- brak aktywnej sesji → operacja tylko na statusie konta.

Wynik końcowy

Stan konta użytkownika zostaje zmieniony, a jego aktywność w systemie jest odpowiednio kontrolowana (blokada, odblokowanie lub rozłączenie).

4. Mapowanie aplikacji na protokół

Funkcja aplikacji	Komunikaty RCMP	Opis
Logowanie użytkownika	LOGIN / LOGIN_OK / LOGIN_ERR	Uwierzytelnienie i wydanie tokenu
Dołączenie do pokoju	JOIN_ROOM / ROOM_EVENT	Zarządzanie pokojami
Opuszczenie pokoju	LEAVE_ROOM / ROOM_EVENT	Aktualizacja uczestników
Wysyłanie wiadomości	SEND_MESSAGE / DELIVER_MESSAGE / MESSAGE_ACK	Niezawodne dostarczanie
Wiadomości prywatne	SEND_MESSAGE / DELIVER_MESSAGE	Komunikacja 1:1
Keep-alive	PING / PONG	Utrzymanie aktywnej sesji
Obsługa błędów	ERROR	Informowanie o błędach
Wylogowanie	BYE / BYE_ACK	Zamknięcie sesji

4.1. Rozszerzenia protokołu

W ramach dalszego rozwoju aplikacji możliwe jest rozszerzenie protokołu o:

HISTORY_REQUEST

Żądanie pobrania historii wiadomości pokoju.

HISTORY_RESPONSE

Odpowiedź serwera zawierająca historię wiadomości.

Rozszerzenie umożliwi:

- odtworzenie historii po reconnect,
- synchronizację klienta,
- pobieranie starszych wiadomości.

5. Wymagania niefunkcjonalne

5.1. Bezpieczeństwo

System musi zapewniać:

- szyfrowanie TLS 1.3,
- integralność wiadomości HMAC-SHA256,
- uwierzytelnienie JWT,
- przechowywanie haseł jako bcrypt,
- ochronę przed replay attack,
- rate limiting.

5.2. Wydajność

Założenia wydajnościowe:

- minimum 100 równoczesnych klientów,
- opóźnienie dostarczenia wiadomości poniżej 200 ms w sieci lokalnej,
- maksymalny rozmiar wiadomości: 64 KB.

5.3. Niezawodność

System powinien:

- wykrywać utratę połączenia,
- wspierać reconnect,
- retransmitować wiadomości,
- wykrywać duplikaty,
- obsługiwać timeouty.

5.4. Skalowalność

Architektura systemu umożliwia:

- oddzielenie modułu autoryzacji,
- przeniesienie bazy danych na osobny serwer,
- dalszy rozwój do architektury wieloserwerowej.

5.5. Logowanie i diagnostyka

System powinien logować:

- błędy protokołu,
- próby logowania,
- reconnect użytkowników,
- błędy HMAC,
- przekroczenia rate limit,
- błędy serwera.

6. Plan implementacji i testowania

6.1. MVP (Minimum Viable Product)

Zakres serwera

- obsługa TCP/TLS,
- LOGIN,
- JOIN_ROOM,
- SEND_MESSAGE,
- DELIVER_MESSAGE,
- MESSAGE_ACK,
- PING/PONG,
- reconnect.

Zakres klienta

- aplikacja CLI,
- logowanie,
- dołączanie do pokoiów,
- wysyłanie wiadomości,
- odbieranie wiadomości,
- reconnect.

Baza danych

- użytkownicy,
- pokoje,
- wiadomości,
- ACL.

6.2. Plan testów

Testy funkcjonalne

- poprawne logowanie,
- wysyłanie wiadomości,
- komunikacja prywatna,
- dołączanie do pokoiów,
- reconnect.

Testy błędów

- niepoprawny JSON,
- nieznany typ wiadomości,
- invalid HMAC,
- expired JWT,
- timeout PONG.

Testy bezpieczeństwa

- brute-force login,
- replay attack,
- duplikaty wiadomości,
- próby wejścia do prywatnego pokoju.

Testy obciążeniowe

- wielu równoczesnych klientów,
- wysoka liczba wiadomości,
- reconnect storm,
- spam wiadomościami.

6.3 Podział pracy

Projekt realizowany jest indywidualnie przez autora.

Autor odpowiada za:

- projekt protokołu RCMP,
- projekt architektury systemu,
- implementację klienta i serwera,
- implementację mechanizmów bezpieczeństwa,
- przygotowanie testów,
- dokumentację projektu.